

C++完整知识体系梳理

一、全书思维导图（基于目录）

mindmap

root((C++全书知识))

第3章 构造数据类型

3.1 枚举 enum

3.2 数组(一维/多维/字符数组字符串)

3.3 指针(定义/字符串/数组/多重/动态内存/函数指针)

3.4 引用

3.5 结构体 struct、联合体 union

3.7 指针常量/常量指针

第4章 C++程序结构

4.1 变量类型(全局/局部/存储类型)

4.2 标识符作用域、可见性、名字空间

4.3 编译预处理(#include、宏、条件编译)

3.4 结构与联合、引用、指针常量

第5章 类与对象(面向对象基础)

5.1 类、对象概念

5.2 构造/析构/拷贝构造函数

5.3 对象使用：指针/引用/数组/this/深浅拷贝

5.4 静态成员(变量/函数)

5.5 友元(友元函数、友元类)

5.6 const 常对象、常成员

5.7 对象内存分配释放

第6章 继承与派生

6.1 继承、派生概念

6.2 三种继承方式(公有/保护/私有)

6.3 派生类构造析构

6.4 多继承、类型兼容

6.5 多继承菱形二义性问题

6.6 虚基类解决二义性

第7章 多态性

7.1 多态概述

7.2 运算符重载

7.3 虚函数(静态/动态联编)

7.4 抽象类、纯虚函数
第 8 章 模板
8.1 函数模板
8.2 类模板

二、分章节详细知识点、核心重点、易错点

第 3 章 构造数据类型

1. 完整知识点

1. 枚举 `enum`

- 枚举类型定义、枚举常量默认从 0 自增，可手动赋值
- 枚举变量只能取枚举列表内常量，本质是 `int` 整型

2. 数组

- 一维数组定义、初始化、遍历；多维数组 (二维为主) 行列存储
- 字符数组、C 风格字符串 `char[]`，`\0` 结束标记
- 数组名是首地址常量，不可修改

3. 指针

- 指针定义、取地址`&`、解引用`*`；指针运算 ± 1 代表移动一个类型长度
- 字符指针操作字符串；指针数组、数组指针区分
- 多重指针`**`；动态内存 `new/delete` 申请释放堆空间
- 指针作函数参数、函数指针

4. 引用 `&`

- 引用是变量别名，定义时必须初始化，不可为空，不可二次绑定
- 常引用 `const &` 保护实参不被修改，适合大对象传参

5. 结构体 `struct` / 联合体 `union`

- `struct`: 成员独立分配内存；`union`: 所有成员共享同一块内存

6. 指针常量 vs 常量指针

- `const int *p`: 常量指针，指向的值不可改，指针本身可改
- `int *const p`: 指针常量，指针地址固定，指向值可改

核心重点

1. 指针与数组的等价访问: `a[i] == *(a+i)`
2. `new/delete` 配对使用, 动态数组 `new T[]` 必须用 `delete[]` 释放
3. 引用和指针的核心区别
4. `union` 内存共享特性, 同一时刻仅一个成员有效

易错点

1. 字符串忘记手动添加结束符 `\0`, 导致越界访问
2. 混淆 `char *a[5]`(指针数组) 与 `char (*a)[5]`(数组指针)
3. 动态内存释放遗漏、重复释放、释放栈局部变量
4. 引用定义不初始化、试图给引用重新绑定其他变量
5. 枚举变量直接赋值超出枚举列表的整数, 编译器仅警告不报错

第 4 章 C++ 程序结构

1. 完整知识点

1. 变量分类
 - 全局变量: 作用域整个文件, 默认 0 初始化, 静态存储区
 - 局部变量: 函数 / 块内, 栈存储, 随机垃圾值; `static` 局部变量仅初始化一次
 - 存储类型: `auto`、`static`、`register`、`extern`
2. 作用域与可见性
 - 块作用域、函数作用域、文件作用域; 内层屏蔽同名外层变量
 - 名字空间 `namespace` 解决命名冲突, `::` 域解析符
3. 编译预处理
 - `#include` 头文件包含; 宏 `#define` 文本替换, 无类型检查
 - 条件编译 `#ifdef` / `#else` / `#endif` 区分调试 / 发布代码

核心重点

1. `static` 修饰变量的生命周期、初始化规则

2. 名字空间 `std` 标准库使用方法
3. 局部同名变量屏蔽全局变量规则

易错点

1. `static` 局部变量误以为每次调用都会重新初始化
2. 宏定义不加括号，运算优先级出错（如 `#define S(a,b) a*b` 传参 `(1+2)*(3+4)` 出错）
3. 块内定义同名变量后，无法在块内访问外层同名变量

第 5 章 类与对象（全书核心基础！！）

1. 完整知识点

1. 面向对象三大特性封装、继承、多态；类是抽象模板，对象是实例
2. 访问权限：`private` 私有（默认）、`protected` 保护、`public` 公有
3. 特殊成员函数
 - 构造函数：同名类名、无返回值，创建对象自动调用，可重载
 - 析构函数：`~类名`，无参无重载，对象销毁自动调用，释放资源
 - 拷贝构造函数：参数为同类引用，对象复制 / 传值 / 返回对象时调用
4. 对象操作
 - 对象指针*、对象引用&、对象数组；`this` 指针指向当前实例
 - 浅拷贝：仅复制地址；深拷贝：重新分配堆内存，解决析构重复释放
5. 静态成员 `static`
 - 静态成员变量：属于类，所有对象共享，类外初始化
 - 静态成员函数：无 `this` 指针，仅可访问静态成员，可 `类名::函数()` 调用
6. 友元
 - 友元函数 / 友元类：突破访问权限，可直接访问类私有成员，破坏封装
7. `const` 常属性
 - 常对象 `const 类名 obj`：仅能调用常成员函数
 - 常成员函数 `void func() const`：不可修改成员变量
 - 常成员变量：仅能在构造初始化列表赋值

核心重点

1. 构造、析构、拷贝构造调用时机与执行顺序
2. this 指针用途、深浅拷贝区别与实现
3. static 静态成员不属于对象，全局唯一
4. const 常对象与常成员函数绑定规则

易错点

1. 构造函数加返回值、析构函数加参数 / 重载
2. 拷贝构造函数参数不用引用，无限递归栈溢出
3. 浅拷贝动态内存导致双重释放崩溃
4. 普通成员函数调用常对象、常成员函数修改成员变量
5. 静态成员函数使用 this 指针（编译报错）
6. 常成员变量在构造函数体内赋值（只能初始化列表）

第 6 章 继承与派生

1. 完整知识点

1. 继承：代码复用，派生类拥有基类全部成员，访问权限受继承方式限制
2. 三种继承方式
 - public 公有继承：基类 public→派生 public, protected→protected
 - protected 保护继承：基类 public/protected→派生 protected
 - private 私有继承：基类所有非私有成员→派生 private
3. 派生类构造与析构
 - 构造顺序：先基类构造 → 对象成员构造 → 派生类自身构造
 - 析构顺序：与构造完全相反；基类析构建议加 virtual 防内存泄漏
4. 多继承：一个派生类继承多个基类
5. 菱形继承（钻石继承）：多继承产生重复基类副本，存在二义性
6. 虚基类 `virtual public`：共享唯一基类副本，解决菱形二义性

核心重点

1. 三种继承方式对基类成员访问权限的修改
2. 派生类构造、析构执行顺序
3. 虚基类解决菱形继承二义性原理

易错点

1. 私有继承误以为外部可访问基类 `public` 成员
 2. 派生类构造函数不会自动调用带参基类构造，必须初始化列表显式调用
 3. 菱形继承不使用虚基类，访问顶层基类成员产生二义性编译错误
 4. 基类析构不加 `virtual`，派生类动态销毁时派生析构不执行，内存泄漏
-

第 7 章 多态性（全书重难点）

1. 完整知识点

1. 多态：同一接口，不同实现；分为静态多态、动态多态
2. 静态多态：编译期确定
 - 函数重载、运算符重载（成员重载 / 友元重载）
3. 动态多态：运行期确定，依赖虚函数
 - 虚函数 `virtual`：基类声明，派生类重写，基类指针 / 引用指向派生对象调用派生版本
 - 静态联编：编译绑定函数地址；动态联编：运行查表 `vtable` 绑定
4. 纯虚函数 `virtual void func() = 0`：无实现，包含纯虚函数的类为抽象类
5. 抽象类：不能实例化对象，仅作基类规范接口；派生类必须实现全部纯虚函数才可实例化

核心重点

1. 虚函数实现动态多态的条件：基类指针 / 引用、虚函数重写
2. 纯虚函数、抽象类作用
3. 运算符重载两种实现方式对比

易错点

1. 派生类重写虚函数时参数 / 返回值不匹配，不构成重写，仅隐藏

- 2. 抽象类试图创建实例，编译报错
- 3. 基类普通函数，基类指针指向派生对象仍调用基类版本，无多态
- 4. 运算符重载改变运算符原有优先级、操作数个数（非法）

三、核心知识点对比表

对比 1：指针 vs 引用

维度	指针 <code>T *p</code>	引用 <code>T &r = obj</code>
本质	独立变量，存储内存地址	变量别名，不独立分配内存
初始化	可空 <code>nullptr</code> ，可后赋值、可重定向	定义时必须绑定变量，不可为空，不可更换绑定
使用	解引用*取值，取地址&p 获取自身地址	直接当作原变量使用，&r 获取原变量地址
内存开销	占 4/8 字节（存储地址）	无额外内存开销
适用场景	动态内存、数组、链表、 可空参数	函数传参避免拷贝、无需 空值场景

对比 2：三种继承方式访问权限

基类成员	public 公有继承	protected 保护继承	private 私有继承
public	派生类 public，外部可访问	派生类 protected，外部不可访问	派生类 private，外部不可访问
protected	派生类 protected，仅派生	派生类 protected	派生类 private

	/ 子类访问		
private	派生类不可直接访问，仅通过基类接口	派生类不可直接访问	派生类不可直接访问

对比 3：静态联编 vs 动态联编

维度	静态联编（编译期多态）	动态联编（运行期多态）
实现方式	函数重载、运算符重载	virtual 虚函数 + 基类指针 / 引用
绑定时机	编译阶段确定调用函数	程序运行时查表确定函数
开销	无运行时开销，效率高	存在虚表查询微小开销
核心特征	和对象实际类型无关	根据指针指向真实对象类型调用对应函数

对比 4：浅拷贝 vs 深拷贝

维度	浅拷贝（默认拷贝构造）	深拷贝（自定义拷贝构造）
堆内存处理	仅复制指针地址，两个对象共享同一块堆内存	新分配独立堆内存，复制内容，内存完全隔离
析构风险	双重释放同一内存，程序崩溃	各自释放自有内存，无冲突
适用场景	无动态堆成员的简单类	包含 new 动态内存成员类

对比 5：const 修饰指针两种写法

写法	含义	可修改内容
----	----	-------

<code>const int *p</code>	常量指针，指向的值只读	p 本身可以指向其他地址，*p 不可修改
<code>int *const p</code>	指针常量，指针地址固定	p 不可指向其他地址，*p 的值可修改

对比 6：构造函数 / 拷贝构造 / 析构函数

函数	格式	调用时机	核心作用
构造函数	类名 (参数)	对象创建时	初始化成员变量、分配资源
拷贝构造	类名 (const 类名 &)	对象复制、函数传值、函数返回对象	复制已有对象生成新对象
析构函数	~ 类名 ()	对象销毁时	释放堆内存、关闭文件等资源清理